PART A

Experiment 3

A.1 Aim: Version Control with Git: Branching, Merging, and Resolving Conflicts

A.2 Prerequisite.

Windows PC, Git installed, basic command prompt knowledge.

A.3 Outcome

Students can initialize a Git repository, create commits, work with branches, merge changes, and resolve merge conflicts.

A.4 Theory

Git is a distributed version control system used to track changes in files and manage different versions of a project. It helps developers maintain project history and work on software collaboratively.

1. Git Repository: A repository is a location where Git stores project files and tracks their changes. The git init command initializes a new Git repository.

2. Git Status: The git status command displays the current state of files, including untracked files and changes ready to be committed.

3. Git Add: The git add command stages files so that their changes can be included in the next commit.

4. Git Commit: A commit records a snapshot of the staged changes in the repository. The git commit command is used to save these changes with a descriptive message.

5. Branching: A branch is an independent line of development in a project. Branches allow developers to work on new features without directly affecting the main branch. The git branch command creates a branch, and git switch is used to move between branches.

6. Merging: Merging combines changes from one branch into another. The git merge command is used to integrate changes from the specified branch into the current branch.

7. Merge Conflict: A merge conflict occurs when Git cannot automatically combine changes, such as when the same line of a file is modified differently in two branches. The conflict must be resolved manually by editing the file, staging the changes, and committing the resolution.

8. Git Log: The git log command displays the commit history of a repository. Thus, Git helps manage project versions, maintain change history, develop features using branches, merge changes, and resolve conflicts efficiently.

Commands

git init
git status
git add .
git commit -m "First Commit"
git branch login
git switch login
git switch main
git merge login
git log

Procedure

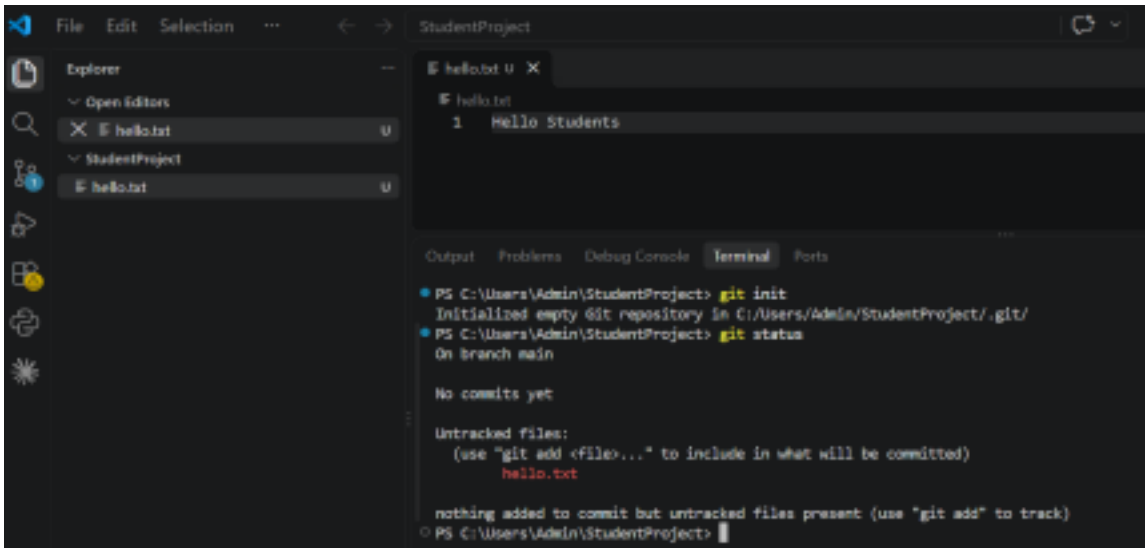
1. Create a project folder and open Terminal.
2. Run: git init
3. Create hello.txt and write 'Hello Students'.
4. Run git status.
5. Run git add .
6. Run git commit -m "First Commit".
7. Create a branch: git branch login
8. Switch: git switch login
9. Edit hello.txt, add 'Login Feature Added'.
10. Commit changes.
11. Switch back to main and merge the login branch.
12. To create a conflict, edit the same line differently in both branches, merge, resolve markers, then commit.

PART B
(PART B: TO BE COMPLETED BY STUDENTS)

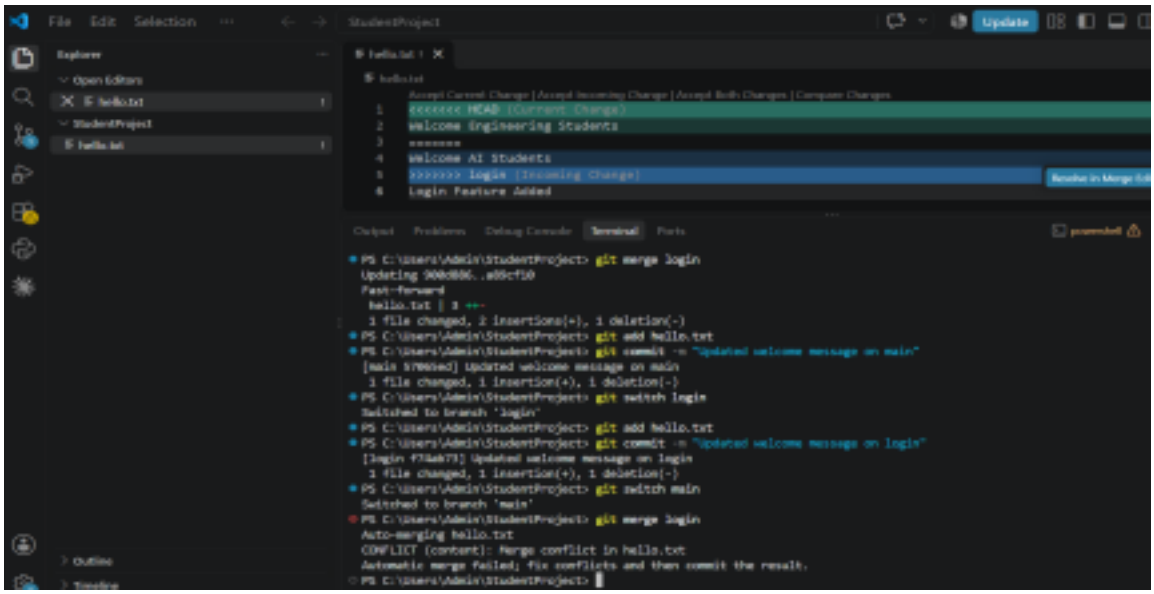
Roll No. A27	Name: Prajwal Ashok Zolage
Class: BE–AI&DS	Batch: A1
Date of Experiment: 04/08/2026	Date of Submission: 04/08/2026

B.1 Code Screenshot:

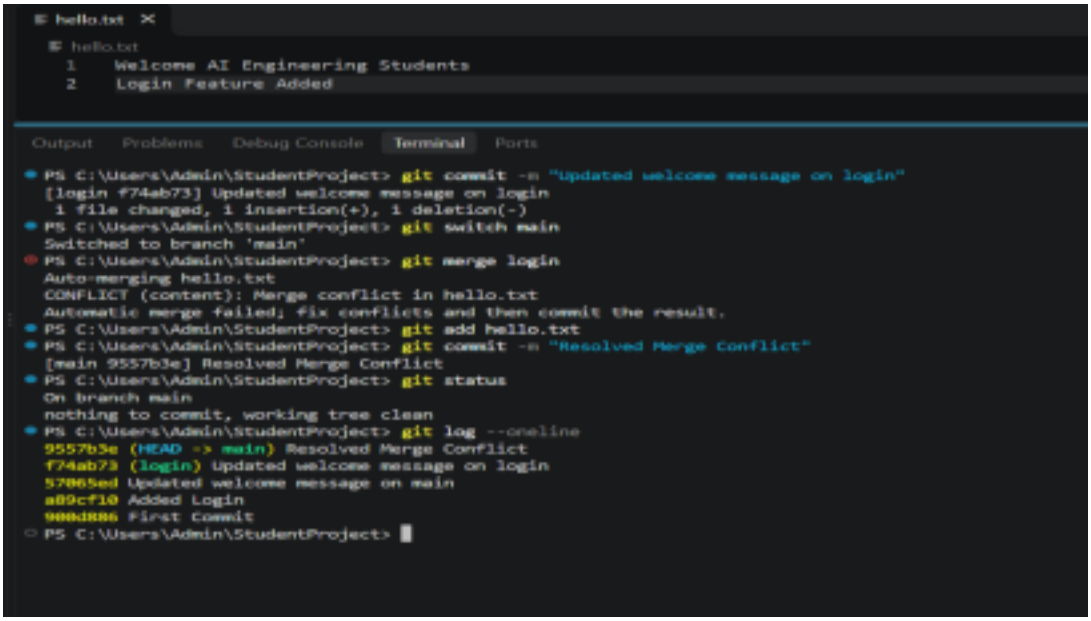
1. Initial hello.txt file and Git status.



2. Merge conflict in hello.txt.

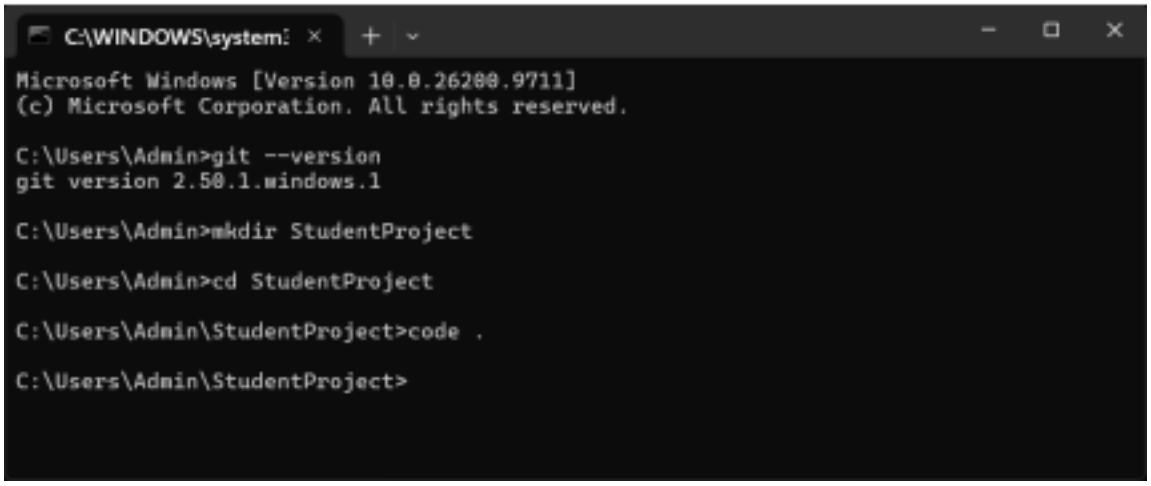


3. Resolved hello.txt file.

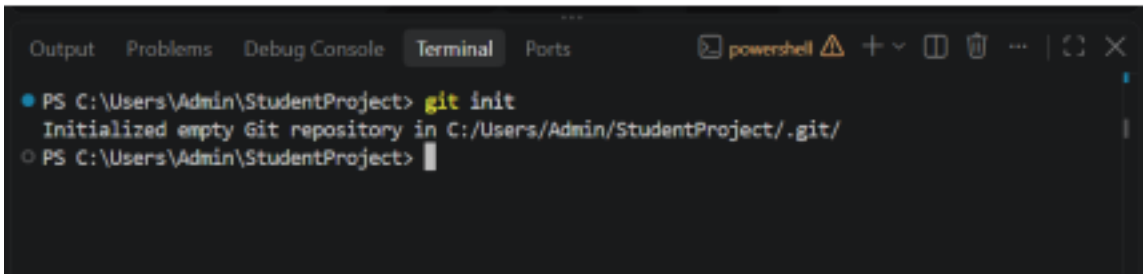


B.2 Input and Output:

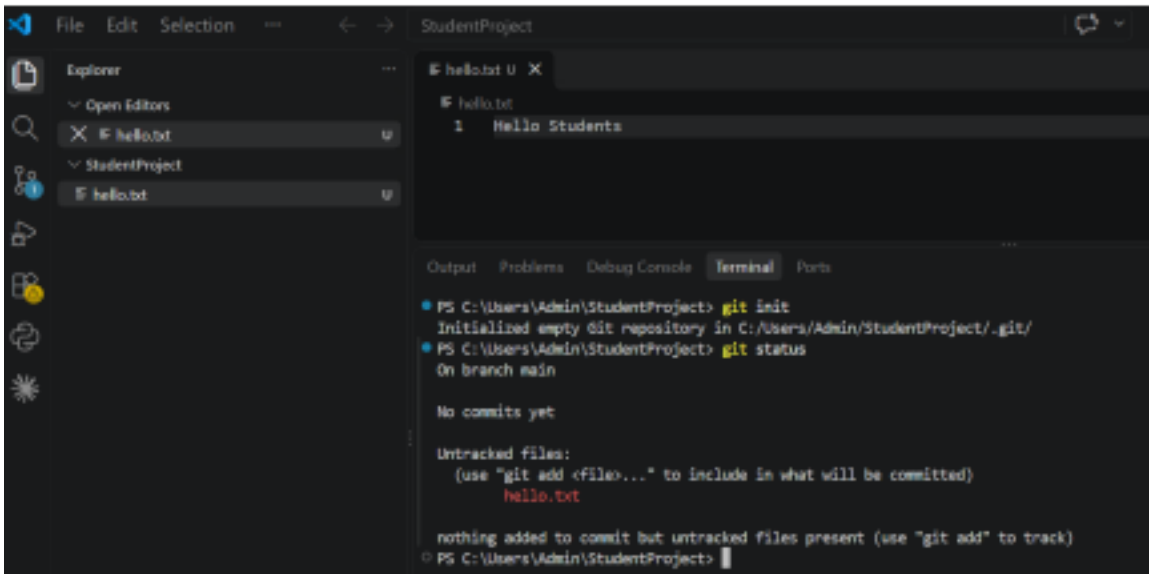
Checking Git version and creating the project folder



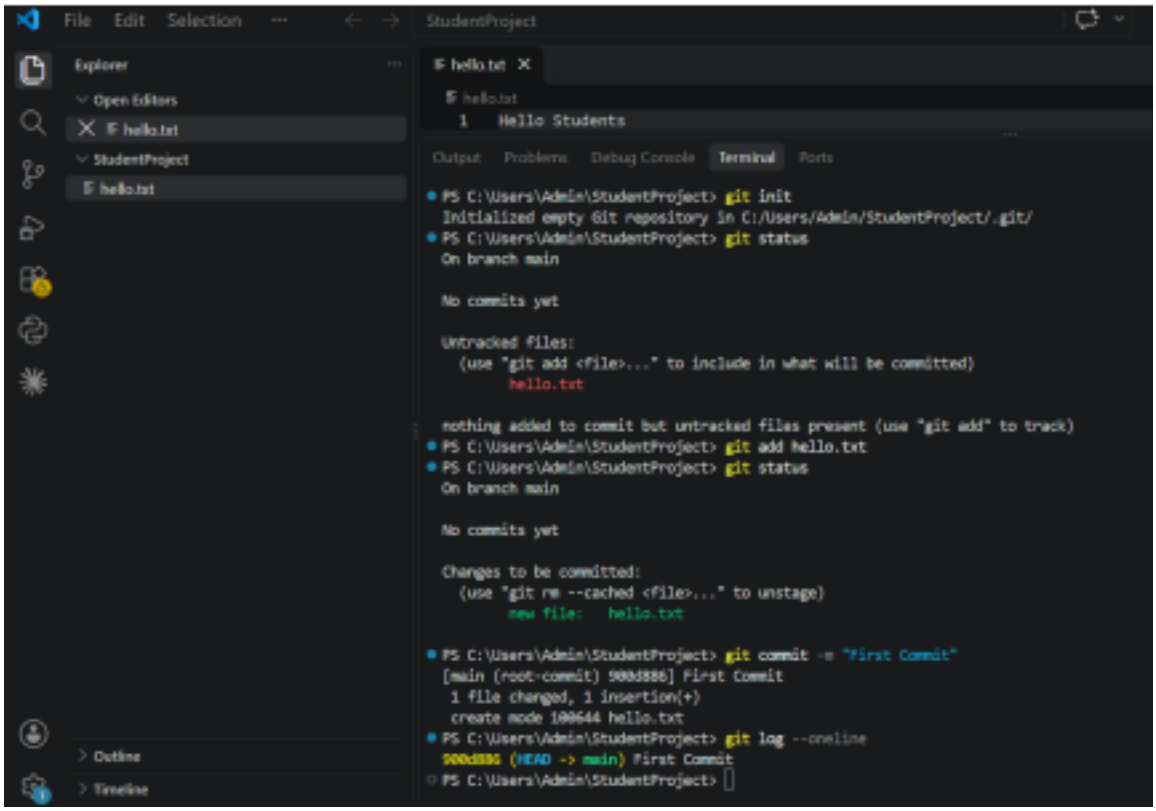
Initializing the Git repository using git init



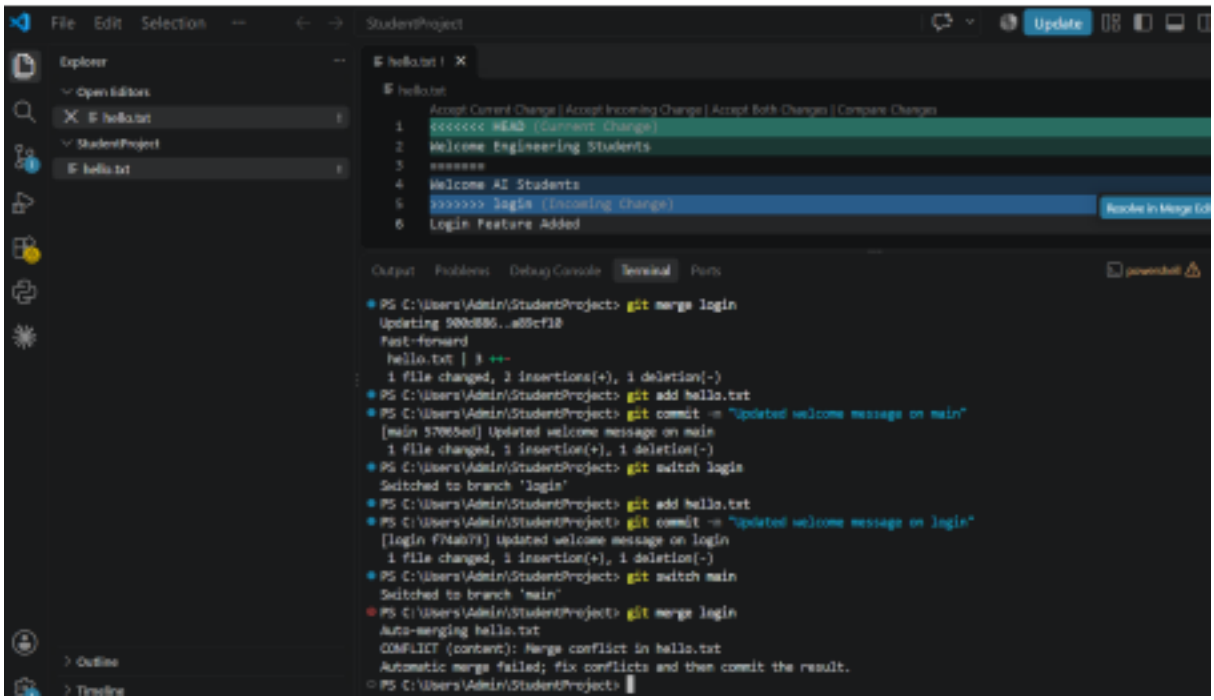
Checking file status using git status



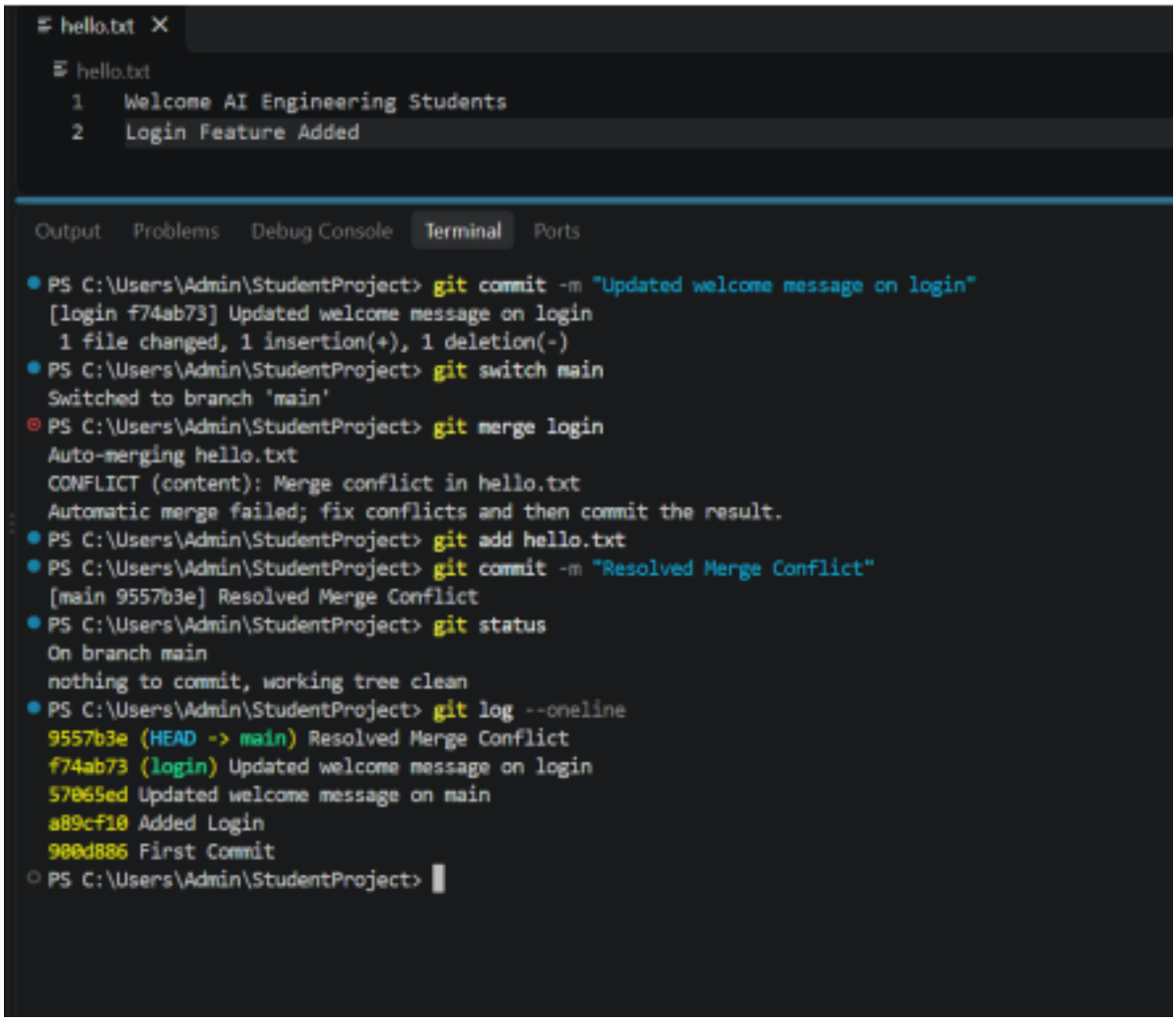
Adding the file, committing changes, and checking Git history



Merging branches and identifying a merge conflict



Resolving the conflict and verifying the clean working tree



B.3 Conclusion:

Git version control was successfully implemented using CMD and Visual Studio Code. The repository was initialized, files were added and committed, branches were created, and changes were merged. A merge conflict was created and resolved successfully. The final Git status confirmed that the working tree was clean.

****